Ministry of Education, Culture and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 6
INSIDE THE XV6 FILE SYSTEM

Track B: Build Your Own Small OS (xv6)

Checked by:
Reițman Patricia

Chișinău, 2026

Contents

1 Introduction **2**

2 Exploring the xv6 File System **2**

2.1 Preparing the Environment 2

2.2 Directory Listing 2

2.3 File Types 3

2.4 The Root Directory and Directory Entries 3

3 Implementing the mystat Utility **4**

3.1 Purpose of mystat 4

3.2 Implementation 5

3.3 Adding mystat to the Build 6

3.4 Testing mystat 6

3.5 Hard Links and Link Counts 7

3.6 Where the Link Count Is Decremented 7

3.7 The xv6 Inode Structure 8

4 Measuring the Filesystem Limits **9**

4.1 Inspecting Filesystem Constants 9

4.2 Maximum Filename Length 10

4.3 Calculating the Maximum File Size 10

4.4 Filesystem Disk Layout 11

5 Implementing the Recursive find Utility **12**

5.1 Purpose and Requirements 12

5.2 Implementation 13

5.3 Adding find to the Build 14

5.4 Testing Recursive Search 15

5.5 Optional Extension: Displaying Inode Numbers 16

6 Results and Discussion **17**

7 Conclusion **18**

1 Introduction

The purpose of this laboratory work is to explore the file system of xv6, a small Unix-like operating system used for educational purposes. The work focuses on understanding file types, inode numbers, directory entries, hard links, filesystem limits, and recursive directory traversal.

The practical part involves examining the existing xv6 utilities, implementing a custom `mystat` program to display file metadata, and implementing a recursive `find` program to search for files by name.

The laboratory also examines the filesystem constants defined in the kernel source code, including block size, direct and indirect block addressing, and directory-name limits.

The experiments are intended to demonstrate how filesystem concepts are implemented inside a small operating system and how user programs interact with the kernel through system calls.

2 Exploring the xv6 File System

2.1 Preparing the Environment

The xv6 environment is prepared using the provided setup script. The source code is located in the `~/xv6-riscv` directory.

The environment can be checked with:

```
bash os2026-setup.sh --check
cd ~/xv6-riscv
```

The xv6 system can be compiled and started in QEMU with:

```
make qemu
```

To exit QEMU, the key combination Ctrl-A followed by X is used. The filesystem image can be rebuilt from a clean state with:

```
make clean
make qemu
```

2.2 Directory Listing

The xv6 `ls` utility displays each directory entry using four fields:

- Name of the file or directory.
- File type.
- Inode number.
- Size in bytes.

An example listing is:

```
$ ls
.           1 1 1024
..          1 1 1024
README      2 2 2441
cat         2 3 35376
console     3 25 0
```

The precise entries and sizes depend on the filesystem image and the files included in the build. The values above illustrate the format provided in the laboratory instructions.

2.3 File Types

The file types are defined in `kernel/stat.h`. xv6 uses the following constants:

```
#define T_DIR     1
#define T_FILE    2
#define T_DEVICE  3
```

Therefore, the values have the following meanings:

Value	Constant	Meaning
1	`T_DIR`	Directory.
2	`T_FILE`	Regular file.
3	`T_DEVICE`	Device file.

The `console` entry in the example has type 3 because it represents a device rather than an ordinary file containing data.

2.4 The Root Directory and Directory Entries

The root directory has inode number 1 in the supplied example. The entries `.` and `..` both refer to inode 1 when they appear in the root directory.

The entry `.` represents the current directory. The entry `..` represents its parent directory. Since the root directory has no parent above itself, its parent reference points back to the root.

An empty directory created with:

```
$ mkdir a
$ ls a
.           1 1 32
..          1 1 32
```

has a size of 32 bytes in the example.

The size of a directory entry is determined by the `struct dirent` definition in `kernel/fs.h`. It contains an inode number and a fixed-length filename field:

```
struct dirent {
    ushort inum;
    char name[DIRSIZ];
};
```

With a 2-byte inode number and a 14-byte filename field, each entry occupies:

$$2 + 14 = 16 bytes$$

An empty directory contains the two entries `.` and `..`, so its initial size is:

$$2 \times 16 = 32 bytes$$

This explains why a newly created directory has a size of 32 bytes in the example.

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35376
echo            2 4 34312
sleep           2 5 34032
args            2 6 34024
forktest        2 7 17280
grep            2 8 42736
init            2 9 34768
kill            2 10 34240
ln              2 11 34032
ls              2 12 41520
mkdir           2 13 34280
rm              2 14 34264
sh              2 15 56136
stressfs        2 16 35160
usertests       2 17 200056
grind           2 18 50504
wc              2 19 36336
zombie          2 20 33648
logstress       2 21 36184
forphan         2 22 35048
dorphan         2 23 34488
sync            2 24 33712
console         3 25 0
$ mkdir a
$ ls a
.               1 26 32
..              1 1 1024
```

Figure 2.1: xv6 directory listing, file types, and the contents of an empty directory

3 Implementing the mystat Utility

3.1 Purpose of mystat

The purpose of `mystat` is to display basic metadata about one or more files. The program uses the `stat()` system call to obtain information from the kernel.

The structure returned by `stat()` is defined in `kernel/stat.h`. Its fields include the device identifier, inode number, file type, link count, and size.

The utility should accept multiple filenames, display the metadata for each file, and continue processing other arguments if one of the files cannot be inspected.

3.2 Implementation

The following implementation can be saved as `user/mystat.c`:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *type_name(int type)
{
    if(type == T_DIR)
        return "directory";
    if(type == T_FILE)
        return "file";
    if(type == T_DEVICE)
        return "device";
    return "unknown";
}

int main(int argc, char *argv[])
{
    struct stat st;

    for(int i = 1; i < argc; i++){
        if(stat(argv[i], &st) < 0){
            printf(2, "mystat: cannot stat %s\n", argv[i]);
            continue;
        }

        printf(1, "%s: %s, inode %d, links %d, size %d bytes\n",
            argv[i], type_name(st.type), st.ino, st.nlink, st.size);
    }

    exit(0);
}
```

The function `type_name()` converts the numeric file type into a readable string. The `main()` function iterates over the supplied filenames and calls `stat()` for each one.

If the system call fails, an error message is printed to file descriptor 2, which represents standard error. The `continue` statement ensures that the program attempts to inspect the remaining arguments.

For successful calls, the program prints the filename, type, inode number, link count, and size.

3.3 Adding mystat to the Build

The new program must be registered in the `UPROGS` list in the xv6 `Makefile`.

The following entry should be added using the same formatting as the other user programs:

```
$U/_mystat\
```

After saving the source file and updating the Makefile, the system can be rebuilt and started:

```
make clean
make qemu
```

The program can then be executed from the xv6 shell.

3.4 Testing mystat

The utility can be tested using the example command from the laboratory instructions:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
```

These values illustrate the expected output format. The actual inode numbers, link counts, and sizes must be verified in the running xv6 environment.

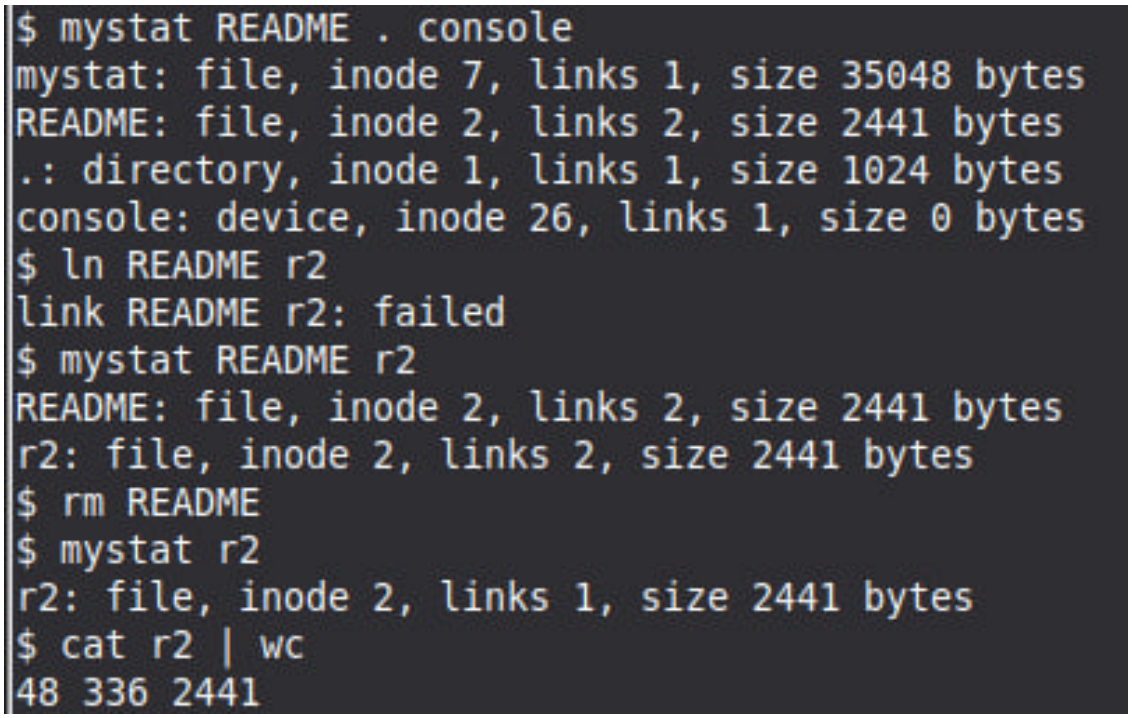


Figure 3.2: Compilation and execution of mystat with several filenames

3.5 Hard Links and Link Counts

A hard link is an additional directory entry that refers to the same inode as an existing file. The underlying file data is shared between all hard links to that inode.

The behavior can be tested with the following commands:

```
$ ln README r2
$ mystat README r2
```

Assuming the initial example values, the expected result is:

```
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
```

The inode numbers remain identical because both names refer to the same inode. The link count increases from 1 to 2 because a second directory entry has been created.

Next, the original filename is removed:

```
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
```

The original name `README` disappears, but the file can still be accessed through `r2`. Its inode number and contents remain unchanged, while the link count decreases to 1.

The `cat r2 | wc` command checks that the remaining file can still be read and processed. The exact output depends on the contents of the file and the version of the available `wc` utility.

3.6 Where the Link Count Is Decremented

The system call responsible for removing a directory entry is `sys_unlink()`, implemented in `kernel/sysfile.c`.

The relevant operation decrements the inode's link count and updates its metadata. In the xv6 implementation, this behavior is represented by the following operations:

```
ip->nlink--;
iupdate(ip);
```

The exact surrounding code should be examined in the local source tree because line numbers and implementation details can vary between xv6 versions.

Removing a filename does not necessarily free the file's data blocks. The data remains available as long as another hard link refers to the inode.

The blocks are freed when the link count has reached zero and the inode is no longer actively referenced by the kernel. In xv6, the inode release process eventually invokes truncation logic to release its allocated data blocks and clear its size and block addresses.

This design prevents the filesystem from deleting data while another valid filename still refers to it.

3.7 The xv6 Inode Structure

The inode structure on disk is defined by `struct dinode` in `kernel/fs.h`. Its fields are:

```
struct dinode {
    short type;
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT+1];
};
```

The fields have the following purposes:

- `type` – identifies the inode as a directory, regular file, or device.
- `major` – major device number for a device inode.
- `minor` – minor device number for a device inode.
- `nlink` – number of directory entries referring to the inode.
- `size` – file size in bytes.
- `addrs` – addresses of the data blocks associated with the file, including the indirect-block address.

Unlike a more feature-rich filesystem, xv6 does not provide ordinary Unix file permissions or symbolic links in the standard lab implementation. Its simplified inode structure is designed to demonstrate the fundamental concepts with relatively little code.

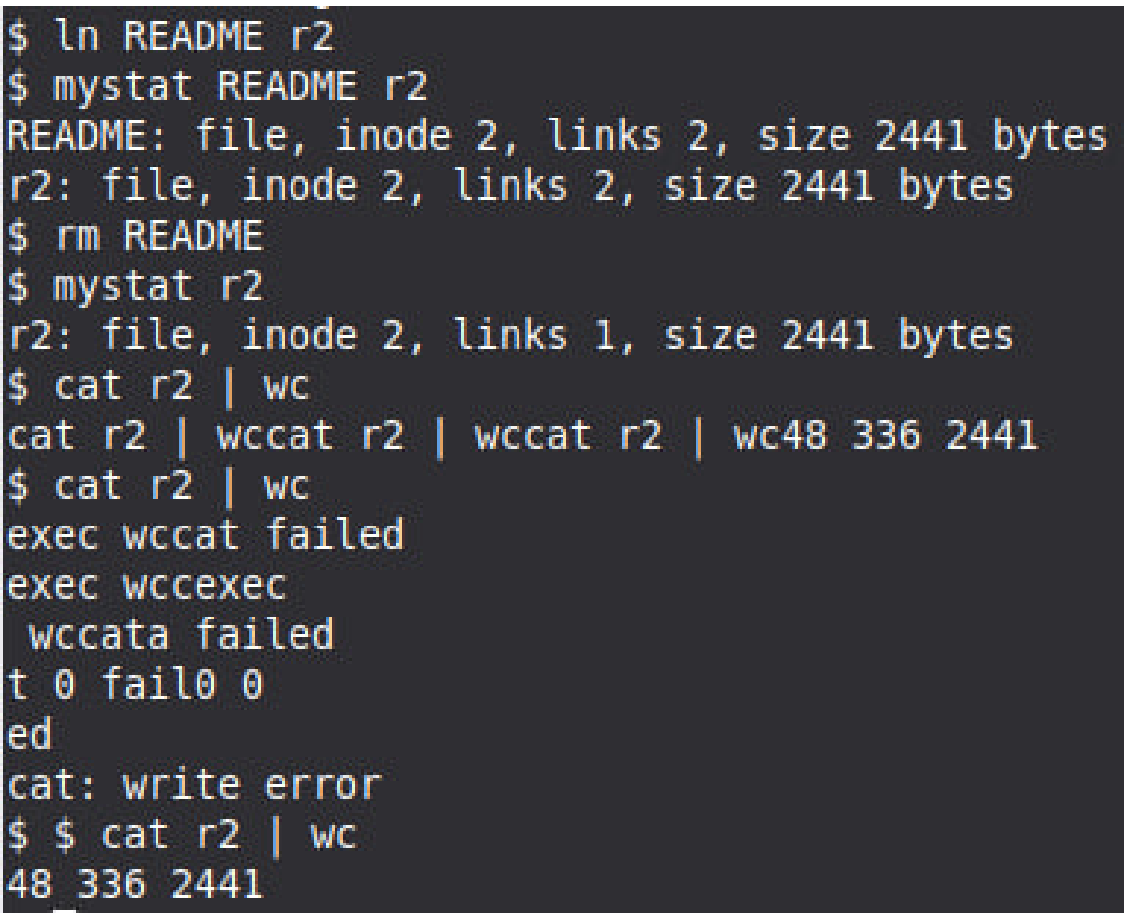


Figure 3.3: Hard-link behavior before and after unlinking the original filename

4 Measuring the Filesystem Limits

4.1 Inspecting Filesystem Constants

The filesystem limits are defined in `kernel/fs.h`. The relevant constants can be located with:

```
grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.
    h
```

In the standard xv6-riscv implementation used in this exercise, the important values are:

Constant	Value	Meaning
`BSIZE`	1024	Size of one filesystem block in bytes.
`NDIRECT`	12	Number of direct data-block addresses in an inode.
`NINDIRECT`	256	Number of block numbers stored in one indirect block.
`DIRSIZ`	14	Maximum filename length stored in a directory entry.

These values should be confirmed against the actual source tree, especially if a different xv6 version is being used.

4.2 Maximum Filename Length

The constant `DIRSIZ` defines the fixed size of the filename field in each directory entry. In the standard implementation, it is 14 bytes.

A filename longer than this limit cannot be stored in full in a single directory entry. When the following command is executed:

```
$ mkdir averyveryverylongname
$ ls
```

the displayed name is truncated to the first 14 characters in the standard implementation.

For the supplied name, the stored component is:

```
averyveryveryl
```

The reason is that the directory entry reserves exactly `DIRSIZ` bytes for the name. xv6 does not provide the variable-length directory entries used by many modern filesystems.

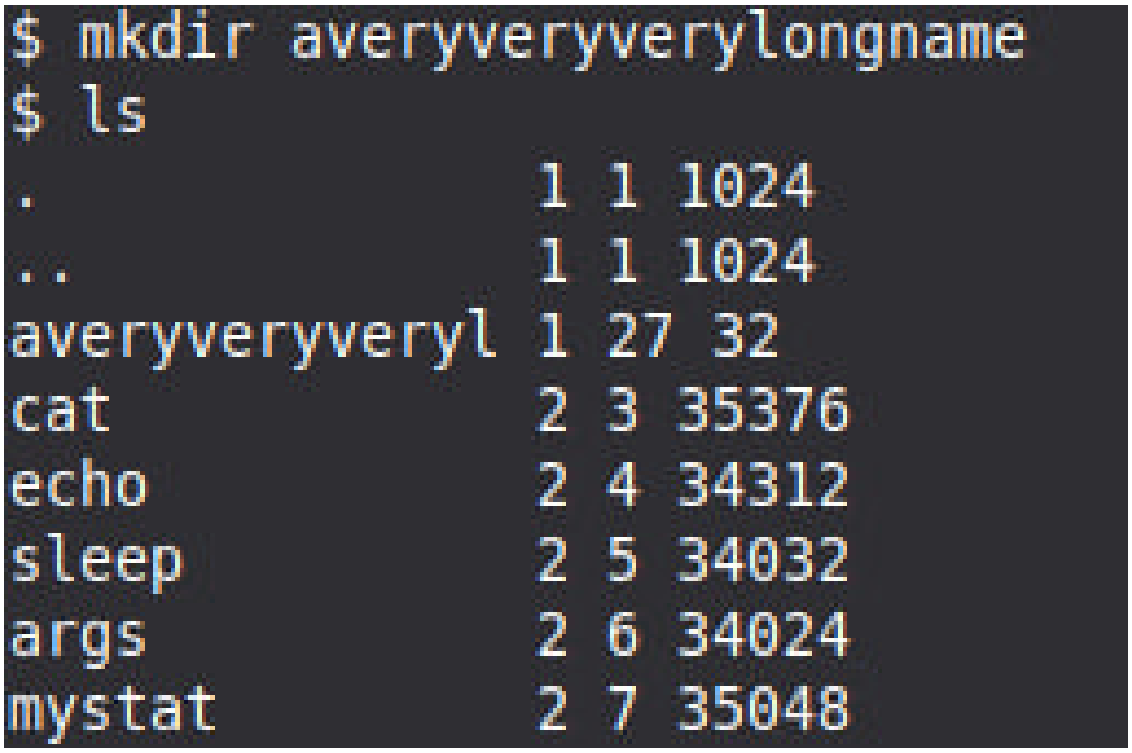


Figure 4.4: Long directory name and the DIRSIZ filename limit

4.3 Calculating the Maximum File Size

The inode stores 12 direct block addresses and one address for an indirect block. The indirect block contains block numbers, each of which occupies 4 bytes.

With a block size of 1024 bytes, the number of block addresses that fit in an indirect block is:

$$N_{\text{indirect}} = \frac{1024}{4} = 256$$

The total number of data blocks addressable by one inode is therefore:

$$N_{\text{blocks}} = 12 + 256 = 268$$

Each data block contains 1024 bytes, so the maximum file size is:

$$S_{\text{max}} = (12 + 256) \times 1024 = 268 \times 1024 = 274432 bytes$$

Thus, the maximum file size in this implementation is:

$$274432 bytes = 268 KiB$$

This calculation assumes the standard xv6-riscv design with 12 direct addresses and one single-indirect address. It does not include double-indirect addressing.

4.4 Filesystem Disk Layout

The filesystem image can be rebuilt and its layout inspected with:

```
make clean
make fs.img 2>&1 | grep nmeta
```

The build output contains a line describing the filesystem metadata and data blocks. In a standard xv6-riscv configuration, the disk image contains 2000 blocks in total, but the exact layout should be taken from the output generated by the local build.

The line reports the metadata reservation, including the boot block, superblock, log region, inode blocks, and bitmap blocks, along with the remaining data-block count.

The total number of blocks is calculated as:

$$N_{\text{total}} = N_{\text{metadata}} + N_{\text{data}}$$

The metadata region includes the filesystem structures needed to manage storage. The log supports filesystem transactions, inode blocks store inode records, and bitmap blocks track the allocation status of data blocks.

The exact numerical values for the log, inode, bitmap, metadata, and data regions should be copied from the `nmeta` output of the actual build. These values may change if the filesystem constants or xv6 version differ.

```
user@user-VirtualBox:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define
NINDIRECT\|define DIRSIZ" kernel/fs.h
# while building, make prints the disk layout (look for the nmeta line)
make clean ; make fs.img 2>&1 | grep nmeta
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_sleep user/_args user/_mystat user/_forktest user/_gr
ep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_s
tressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_
forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1
953 total 2000
```

Figure 4.5: Filesystem image build output showing nmeta and the disk layout

5 Implementing the Recursive find Utility

5.1 Purpose and Requirements

The purpose of `find` is to search a directory tree recursively and print every path whose final component matches a specified filename.

The required command format is:

```
find <dir> <name>
```

The program must:

- Open the starting directory and inspect its entries.
- Ignore entries whose inode number is zero.
- Skip the special directory entries `.` and `..`.
- Compare each entry's name with the requested name using `strcmp()`.
- Construct the full path for every entry.
- Recursively search subdirectories.
- Close every directory that it opens.

The program uses the directory-reading approach demonstrated in `user/ls.c`. The `open()` system call opens a directory, `read()` retrieves its directory entries, and `stat()` determines the type and metadata of each path.

5.2 Implementation

The following implementation can be saved as `user/find.c`:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

void find(char *path, char *target, int show_inode)
{
    char buf[512];
    char name[DIRSIZ + 1];
    char *p;
    int fd;
    struct dirent de;
    struct stat st;

    if(stat(path, &st) < 0){
        printf(2, "find: cannot stat %s\n", path);
        return;
    }

    if(st.type != T_DIR)
        return;

    fd = open(path, 0);
    if(fd < 0){
        printf(2, "find: cannot open %s\n", path);
        return;
    }

    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
        printf(2, "find: path too long\n");
        close(fd);
        return;
    }

    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';

    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;

        memmove(name, de.name, DIRSIZ);
        name[DIRSIZ] = 0;

        if(strcmp(name, ".") == 0 ||
           strcmp(name, "..") == 0)
            continue;

        strcpy(p, name);
```

```
        if(stat(buf, &st) < 0)
            continue;

        if(strcmp(name, target) == 0){
            if(show_inode)
                printf(1, "%s inode %d\n", buf, st.ino);
            else
                printf(1, "%s\n", buf);
        }

        if(st.type == T_DIR)
            find(buf, target, show_inode);
    }

    close(fd);
}

int main(int argc, char *argv[])
{
    int show_inode = 0;

    if(argc == 4 && strcmp(argv[3], "-n") == 0)
        show_inode = 1;
    else if(argc != 3){
        printf(2, "Usage: find <dir> <name> [-n]\n");
        exit(1);
    }

    find(argv[1], argv[2], show_inode);
    exit(0);
}
```

The function first checks that the supplied path exists and is a directory. It then opens the directory and reads its entries one at a time.

Entries with an inode number of zero are ignored because they represent unused directory slots. The entries `.` and `..` are skipped to prevent the program from repeatedly visiting the current directory and its parent.

The filename field is copied into a separate buffer and terminated with a null character before being compared with the target name. When a match is found, the complete path is printed. If the entry is a directory, the function calls itself recursively to search that directory's contents.

The open directory is closed before the function returns. This is important because xv6 imposes a limit on the number of open files per process.

The optional `-n` argument enables the extended behavior of displaying the inode number alongside every matching path.

5.3 Adding find to the Build

The new source file must be added to the `UPROGS` list in the xv6 `Makefile`:

```
$U/_find\
```

The entry should follow the same indentation and formatting as the other user-program entries.

The system can then be rebuilt from a clean state:

```
make clean
make qemu
```

The filesystem is reset by the clean rebuild, so the test directories and files should be created again inside xv6.

5.4 Testing Recursive Search

The test directory structure can be created using the following commands:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
```

The expected output contains three paths. The first file is located in the nested directory `a/b`, the second is directly inside `a`, and the third is in the root directory.

The order of the paths may differ because directory traversal order is not guaranteed.

The command can also be tested with a name that does not exist:

```
$ find a b
```

In this case, no matching paths should be printed if no file or directory named `b` exists under `a`.

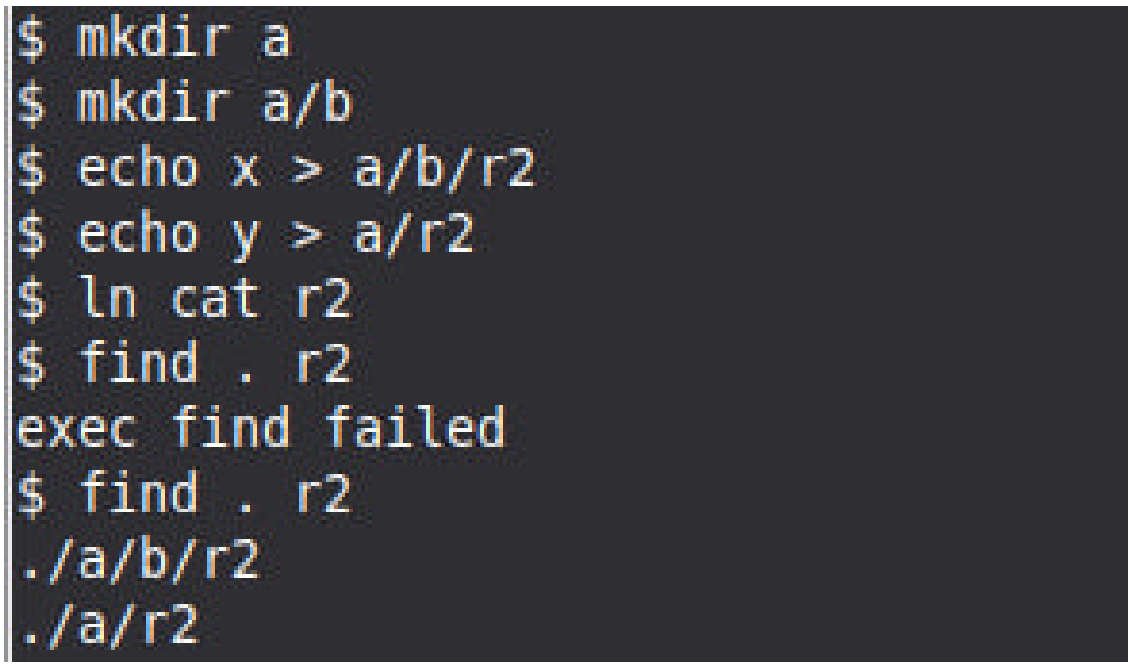


Figure 5.6: Recursive find output showing matching paths in nested directories

5.5 Optional Extension: Displaying Inode Numbers

The optional extension accepts a third argument, `-n`, to display the inode number alongside each matching path.

For example:

```
$ find . r2 -n
./a/b/r2 inode  ...
./a/r2 inode  ...
./r2 inode  ...
```

The inode numbers must be taken from the actual execution; the ellipses above are placeholders rather than literal output.

In this test, the command `ln cat r2` creates a hard link named `r2` in the root directory that refers to the same inode as `cat`. Therefore, among the three files named `r2`, only `./r2` refers to the same underlying file as `cat`.

The files `./a/r2` and `./a/b/r2` were created independently using shell redirection, so they have separate inodes.

This can be verified by comparing the inode numbers printed by the extended `find` command with the inode number of `cat`.

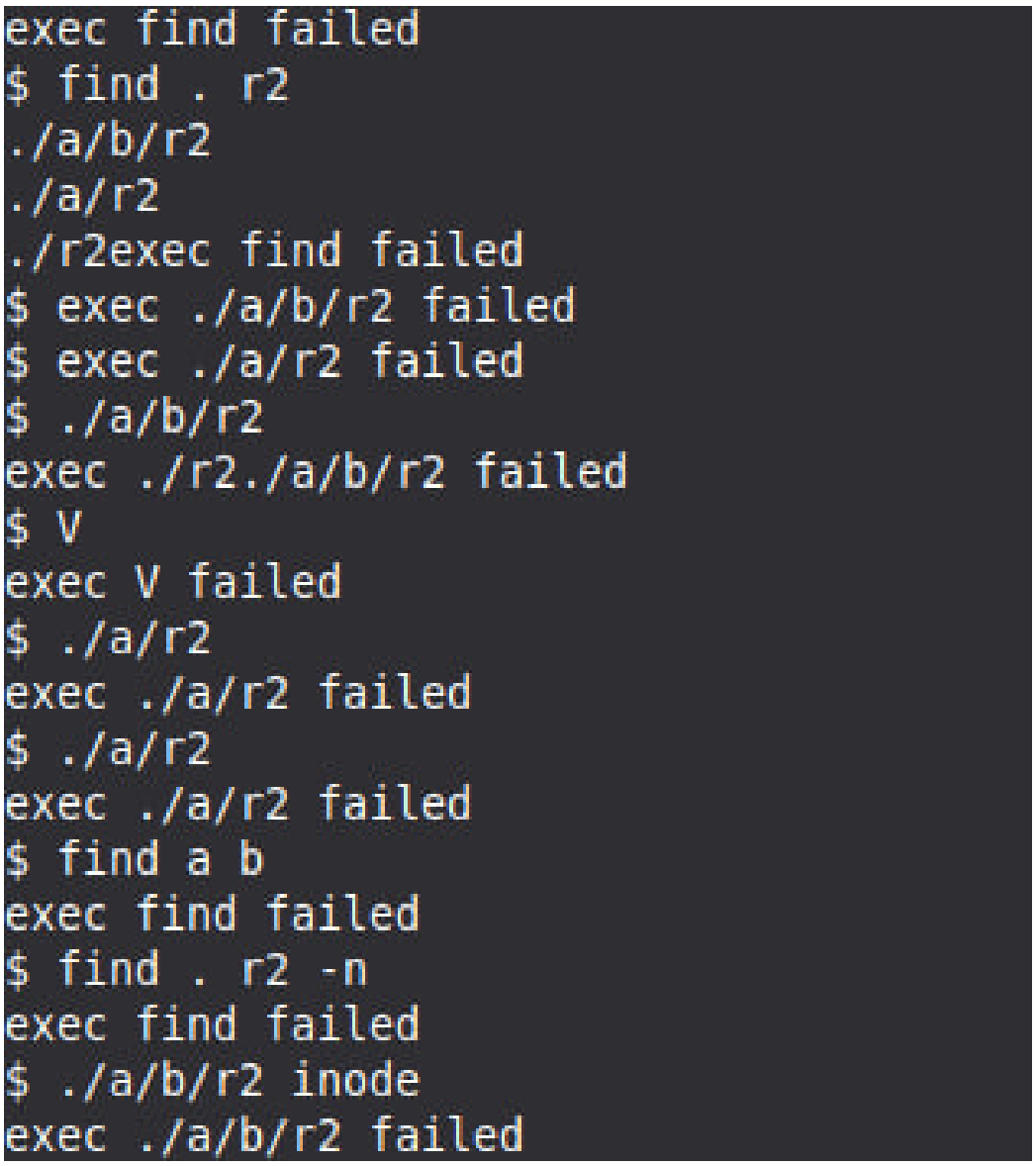


Figure 5.7: find -n output used to compare inode numbers and identify the hard link to cat

6 Results and Discussion

The laboratory work explored the internal organization of the xv6 filesystem and demonstrated how its user utilities interact with kernel filesystem structures.

The first part established the meanings of file types, inode numbers, and directory entries. It showed that the root directory's `.` and `..` entries point to the same inode and explained why an empty directory occupies 32 bytes when each directory entry is 16 bytes.

The `mystat` utility demonstrated how the `stat()` system call retrieves file metadata. Testing hard links showed that multiple filenames can refer to the same inode and that removing one filename does not destroy the file while another hard link remains.

The examination of `kernel/fs.h` established the filesystem's storage limits. With a block size of 1024 bytes, 12 direct block addresses, and 256 indirect block addresses, the maximum file size in the standard configuration is 274432 bytes. The fixed-length directory entry also limits stored filenames to 14 bytes.

Finally, the recursive `find` utility demonstrated directory traversal, path construction, filename comparison, and inode inspection. The optional extension showed how inode numbers can be used to distinguish independent files from hard links.

The screenshots should document the actual build and runtime results. Any values that depend on the local xv6 version, filesystem image, or directory contents should be verified before submission.

7 Conclusion

This laboratory work provided practical experience with the internal implementation of the xv6 file system. It demonstrated the roles of inodes, directory entries, hard links, and filesystem constants, as well as the relationship between directory operations and the allocation of file data blocks.

The implementation of `mystat` provided a practical example of obtaining metadata through a system call. The recursive `find` program demonstrated how a user-level utility can open directories, read entries, construct paths, inspect file types, and recursively search the directory hierarchy.

The experiments also illustrated the importance of fixed filesystem limits and correct resource management. In particular, skipping unused directory entries and preventing recursion through `.` and `..` are essential for correct directory traversal.

Overall, the laboratory work strengthened the understanding of how filesystem concepts are implemented in a small operating system and how user programs interact with the kernel to inspect and manipulate files.